

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

Звіт

**Про виконання лабораторної роботи №3 з Веб програмування на стороні
сервера**

Робота з Node.js, нрм. Практика програмування на JavaScript

Виконав:

Студент групи ФЕІ-23

Лесів Тарас Ігорович

Перевірив:

доц. Чмихало Олег Сергійович

Львів 2026

Лабораторна робота №3

Мета: навчитися створювати npm-проект, установлювати залежності, працювати з аргументами командного рядка засобами Commander, читати вкладені дані з JSON-файлу та коректно обробляти помилки.

Варіант 1. Предметна область: «Розклад занять». Потрібно реалізувати заняття за обраний день; пошук занять викладача з можливістю залишити лише дистанційні; розклад чисельника або знаменника з урахуванням щотижневих занять.

Хід роботи

Для лабораторної роботи я підготував окремий репозиторій:

<https://github.com/TarasLesiv/lab3-schedule-cli>

Усі команди я виконував у Git Bash (MINGW64) на Windows 11.

Спочатку я перевіряв установлені програми (див. Додаток А):

```
$ node --version
v24.21.0
$ npm --version
11.19.0
$ git --version
git version 2.55.0.windows.5
$ jq --version
jq-1.8.2
```

Проект створено й залежність установлено командами:

```
$ npm init -y
$ npm install commander
$ npm list commander --depth=0
|-- commander@14.0.3
```

Commander є єдиним стороннім пакетом; файл читається вбудованим модулем node:fs.

У package.json я додав тип модулів ESM і сценарії start та test.

Основний зміст файла:

```
{
  "name": "lab3-schedule-cli",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "start": "node index.js",
```

```
    "test": "node --test"
  },
  "dependencies": { "commander": "^14.0.3" }
}
```

Структура проєкту має такий вигляд:

```
lab3-schedule-cli/
├─ .gitignore
├─ data.json
├─ index.js
├─ package.json
├─ package-lock.json
├─ README.md
├─ report-lab3.docx
├─ docs/
└─ test/cli.test.js
```

Файл `schedule.txt` із попередньої роботи перейменовано на `data.json`; значення полів не змінювалися. Документ містить групу, курс і масив `schedule`. Кожен день має вкладений масив `lessons`, а кожна пара – об'єкти `numerator` і `denominator` або `null`. Синтаксична перевірка дала код 0 (див. Додаток А):

```
$ jq empty data.json
$ echo $?
0
```

До першого коміту я задав локальне ім'я автора «Тарас Лесів lab3».

Історія містить більше п'яти окремих комітів (див. Додаток А):

```
ffd028d docs: add complete laboratory report
9e09520 docs: add usage guide and command outputs
4e0fb5c test: add CLI integration tests
ada7bee feat: implement schedule CLI commands
328e630 data: add validated schedule JSON
e872a64 chore: initialize npm project and add commander
```

Програма має назву `schedule-cli`, опис, версію 1.0.0 та україномовну довідку. Глобальна опція `-f, --file <path>` працює з кожною командою. Без неї читається `data.json`; інший файл можна передати перед або після команди:

```
$ node index.js --file other.json list
$ node index.js list --file other.json
```

Серед загальних можливостей є:

1. Перелік. Команда `list` показує стислий список днів, а опція `--limit` обмежує їх кількість (див. Додаток В):

```
$ node index.js list --limit 3
Група: ФЕІ-23 | Курс: 2
Показано днів: 3 із 5
```

Основним елементом обрано день, оскільки саме дні є елементами верхнього масиву `schedule`. Для кожного дня підраховується кількість непорожніх пар чисельника і знаменника.

2. Один елемент. Команда `show` має обов'язковий аргумент `<day>` і друкує весь знайдений день у форматі JSON (див. Додаток В):

```
$ node index.js show Понеділок
{
  "day": "Понеділок",
  "lessons": [ ... ]
}
```

Назва дня зрозуміліша за числовий індекс і не залежить від порядку елементів у масиві. Пошук виконується без урахування регістру.

3. Команда `field` має два обов'язкові аргументи: день і крапковий шлях до поля. Індекс масиву також є частиною шляху (див. Додаток С):

```
$ node index.js field Понеділок lessons.0.numerator.subject
Математичний аналіз
$ node index.js field Понеділок lessons.2.numerator
null
```

Для перевірки кожного фрагмента шляху використано `Object.hasOwnProperty()`. Тому наявне значення `null` успішно виводиться, а відсутнє поле дає повідомлення про помилку та код завершення 4. Я не використовував `JSONPath`, бо це вимагало б додаткового стороннього пакета.

Можливості варіанта 1:

1. Заняття за обраний день. Команда `day` приймає обов'язковий аргумент `<day>`. Опція `--week` може мати значення `numerator`, `denominator` або `both`; стандартне значення – `both` (див. Додаток D):

```
$ node index.js day Четвер
День: Четвер | Режим: both
...
Знайдено занять: 3
```

Якщо об'єкти `numerator` і `denominator` однакові, пара є щотижневою та в режимі `both` друкується один раз із позначкою «щотижня». Одна команда з опцією обрана замість трьох майже однакових команд для різних типів тижня.

2. Заняття викладача. Команда `teacher` приймає обов'язковий аргумент `<name>`. Дозволено вказати частину прізвища, а прапорець `--remote` залишає тільки об'єкти з `isRemote: true`:

```
$ node index.js teacher Чмихало --remote
... Веб-програмування (Практична) ... дистанційно
Знайдено занять: 1
```

3. Команда `week` приймає обов'язковий аргумент `numerator` або `denominator`. Опція `--day <day>` за потреби обмежує результат одним днем (див. Додаток Е):

```
$ node index.js week denominator --day Четвер
Розклад: знаменник
Четвер | 2 пара | 10:10-11:30 | щотижня
Четвер | 3 пара | 11:50-13:10 | знаменник
Знайдено занять: 2
```

Щотижнева пара входить до обох типів тижня. Окреме поле `weekly` до `data.json` не додавалося, тому що наявна структура вже подає таке заняття однаковими об'єктами чисельника і знаменника.

Інтерфейс програми

Засіб	Обране рішення	Розглянута альтернатива
<code>--file <path></code>	глобальна опція, <code>data.json</code> за замовчуванням	дублювати опцію в кожній команді
<code>list --limit</code>	необов'язкова опція з додатним числом	обов'язковий аргумент кількості
<code>show <day></code>	назва дня без урахування регістру	нестійкий числовий індекс
<code>field <path></code>	універсальна крапкова нотація	окрема опція для кожного поля
<code>day --week</code>	одна команда, три режими	три окремі команди
<code>teacher --remote</code>	пошук за частиною та логічний прапорець	точний збіг і текстове <code>true/false</code>
<code>week <type></code>	тип тижня – обов'язковий аргумент	дві команди <code>week-numerator/week-denominator</code>

Далі я перевіряв усі команди, аргументи, опції та прапорці, а також глобальну опцію шляху. Для очікуваних помилок стек викликів не виводиться. Застосовано такі коди завершення: 0 – успіх; 1 – помилка файла, JSON або структури; 2 – неправильне значення параметра; 4 – елемент чи поле не знайдено.

Перевірка помилкових викликів (див. Додаток F):

```
$ node index.js --file missing.json list      # код 1
$ node index.js --file invalid.json list      # код 1
$ node index.js show Неділя                  # код 4
$ node index.js field Понеділок lessons.99.x # код 4
$ node index.js list --limit abc              # код 2
$ node index.js show                          # ненульовий код
$ node index.js list --unknown                # ненульовий код
```

Для додаткової перевірки я створив 16 інтеграційних тестів на вбудованому модулі node:test. Вони охоплюють шість можливостей, довідку, глобальну опцію --file, null, відсутнє поле та всі помилки з методички:

```
$ npm test
# tests 16
# pass 16
# fail 0
```

Після успішного тестування я видалив каталог залежностей і відновив його:

```
$ rm -rf node_modules
$ npm install
$ npm test
```

Результат залишився тим самим. Це підтверджує, що node_modules не потрібно зберігати в Git: усі залежності відновлюються за package.json і package-lock.json.

Повний текст index.js наведено в Додатку G. У репозиторії також містяться data.json, автоматичні тести, README та відповіді на теоретичні питання.

Висновок: На даній лабораторній роботі я зрозумів роботу Node.js, npm, Commander і обробку JSON із командного рядка. Найскладнішим було правильно опрацювати вкладені поля та помилки; частина з великою кількістю перевірок здалася трохи громіздкою, але загалом завдання було корисним для розуміння роботи CLI-програм.

Додаток А

```
User@DESKTOP-8NT8CAA MINGW64 ~/lab3-schedule-cli
$ node --version
v24.21.0

User@DESKTOP-8NT8CAA MINGW64 ~/lab3-schedule-cli
$ npm --version
11.19.0

User@DESKTOP-8NT8CAA MINGW64 ~/lab3-schedule-cli
$ git --version
git version 2.55.0.windows.5

User@DESKTOP-8NT8CAA MINGW64 ~/lab3-schedule-cli
$ jq --version
jq-1.8.2

User@DESKTOP-8NT8CAA MINGW64 ~/lab3-schedule-cli
$ npm list commander --depth=0
lab3-schedule-cli@1.0.0 C:\Users\Taras Lesiv\lab3-schedule-cli
`-- commander@14.0.3

User@DESKTOP-8NT8CAA MINGW64 ~/lab3-schedule-cli
$ jq empty data.json

User@DESKTOP-8NT8CAA MINGW64 ~/lab3-schedule-cli
$ echo $?
0
```

Рисунок А1 – Версії Node.js, npm, Git, jq і Commander; перевірка data.json.

```
User@DESKTOP-8NT8CAA MINGW64 ~/lab3-schedule-cli
$ git config --local user.name
Тарас Лесів lab3

User@DESKTOP-8NT8CAA MINGW64 ~/lab3-schedule-cli
$ git log --oneline
ffd028d docs: add complete laboratory report
9e09520 docs: add usage guide, theory and command outputs
4e0fb5c test: add CLI integration tests
ada7bee feat: implement schedule CLI commands
328e630 data: add validated schedule JSON
e872a64 chore: initialize npm project and add commander
```

Рисунок А2 – Локальне ім'я користувача Git та історія комітів.

Додаток В

```
User@DESKTOP-8NT8CAA MINGW64 ~/lab3-schedule-cli
$ node index.js list --limit 3
Група: ФЕТ-23 | Курс: 2
Показано днів: 3 із 5
1. Понеділок: часових слотів – 3, чисельник – 2, знаменник – 3
2. Вівторок: часових слотів – 3, чисельник – 3, знаменник – 2
3. Середа: часових слотів – 3, чисельник – 3, знаменник – 1
```

Рисунок В1 – Стислий перелік днів з опцією --limit.

```
User@DESKTOP-8NT8CAA MINGW64 ~/lab3-schedule-cli
$ node index.js show Понеділок
{
  "day": "Понеділок",
  "lessons": [
    {
      "pair": 1,
      "time": "08:30-9:50",
      "numerator": {
        "subject": "Математичний аналіз",
        "type": "Лекція",
        "teacher": "Цаповська Ж.Я.",
        "room": "1/Б",
        "isRemote": false
      },
      "denominator": {
        "subject": "Математичний аналіз",
        "type": "Практична",
        "teacher": "Трищ Б.О.",
        "room": "209/Т",
        "isRemote": false
      }
    },
    {
      "pair": 2,
      "time": "10:10-11:30",
      "numerator": {
        "subject": "ООП",
        "type": "Лабораторна",
        "teacher": "Берізка І.С.",
        "room": "11/Б",
        "isRemote": false
      },
      "denominator": {
        "subject": "Бази даних",
        "type": "Лабораторна",
        "teacher": "Гусак О.В.",
        "room": "11/Б",
        "isRemote": false
      }
    },
    {
      "pair": 3,
      "time": "11:50-13:10",
      "numerator": null,
      "denominator": {
        "subject": "Англійська мова",
        "type": "Практична",
        "teacher": "Довбенко Л.В.",
        "room": "123/Т",
        "isRemote": false
      }
    }
  ]
}
```

Рисунок В2 – Повний об'єкт дня, отриманий командою show.

Додаток С

```
User@DESKTOP-8NT8CAA MINGW64 ~/lab3-schedule-cli
$ node index.js field Понеділок lessons.0.numerator.subject
Математичний аналіз

User@DESKTOP-8NT8CAA MINGW64 ~/lab3-schedule-cli
$ node index.js field Понеділок lessons.2.numerator
null

User@DESKTOP-8NT8CAA MINGW64 ~/lab3-schedule-cli
$ node index.js field Понеділок lessons.99.subject
Помилка: Поле за шляхом «lessons.99.subject» не існує.
$ echo $?
4
```

Рисунок С1 – Вкладене поле, значення null і помилка відсутнього поля.

Додаток D

```
User@DESKTOP-8NT8CAA MINGW64 ~/lab3-schedule-cli
$ node index.js day Четвер
Група: ФЕІ-23 | Курс: 2
День: Четвер | Режим: both
Четвер | 1 пара | 08:30-9:50 | чисельник
    00П (Лабораторна); Хвишун І.О.; ауд. 11/Б; очно
Четвер | 2 пара | 10:10-11:30 | щотижня
    Математичний аналіз (Практична); Цаповська Ж.Я.; ауд. 209/Т; очно
Четвер | 3 пара | 11:50-13:10 | знаменник
    Англійська мова (Практична); Довбенко Л.В.; ауд. 9/Б; очно
Знайдено занять: 3
```

Рисунок D1 – Заняття за четвер з урахуванням щотижневої пари.

Додаток E

```
User@DESKTOP-8NT8CAA MINGW64 ~/lab3-schedule-cli
$ node index.js teacher Чмихало --remote
Група: ФЕІ-23 | Курс: 2
Викладач: Чмихало | лише дистанційні
Вівторок | 2 пара | 10:10-11:30 | знаменник
    Веб-програмування (Практична); Чмихало О.С.; ауд. 1/Б; дистанційно
Знайдено занять: 1

User@DESKTOP-8NT8CAA MINGW64 ~/lab3-schedule-cli
$ node index.js week denominator --day Четвер
Група: ФЕІ-23 | Курс: 2
Розклад: знаменник
Четвер | 2 пара | 10:10-11:30 | щотижня
    Математичний аналіз (Практична); Цаповська Ж.Я.; ауд. 209/Т; очно
Четвер | 3 пара | 11:50-13:10 | знаменник
    Англійська мова (Практична); Довбенко Л.В.; ауд. 9/Б; очно
Знайдено занять: 2
```

Рисунок E1 – Дистанційні заняття викладача та розклад знаменника.

Додаток F

```
User@DESKTOP-8NT8CAA MINGW64 ~/lab3-schedule-cli
$ node index.js --file missing.json list
Помилка: Файл «missing.json» не знайдено.
$ echo $?
1

User@DESKTOP-8NT8CAA MINGW64 ~/lab3-schedule-cli
$ node index.js list --limit abc
Помилка: Опція --limit очікує додатне ціле число, отримано «abc».
$ echo $?
2

User@DESKTOP-8NT8CAA MINGW64 ~/lab3-schedule-cli
$ node index.js show
Помилка: Пропущено обов'язковий аргумент <day>.
Виконайте «node index.js --help», щоб переглянути довідку.
$ echo $?
1

User@DESKTOP-8NT8CAA MINGW64 ~/lab3-schedule-cli
$ node index.js list --unknown
Помилка: Невідома опція «--unknown».
Виконайте «node index.js --help», щоб переглянути довідку.
$ echo $?
1
```

Рисунок F1 – Обробка помилкових викликів і коди завершення.

```
User@DESKTOP-8NT8CAA MINGW64 ~/lab3-schedule-cli
$ npm test

> lab3-schedule-cli@1.0.0 test
> node --test

TAP version 13
... yci 16 підтестів мають стан ok ...
---
  duration_ms: 50.654252
  ...
1..16
# tests 16
# suites 0
# pass 16
# fail 0
# cancelled 0
# skipped 0
# todo 0
# duration_ms 945.591912
```

Рисунок F2 – Результат автоматичного тестування: 16 тестів пройдено.

Додаток G

Повний текст файлу index.js

```
#!/usr/bin/env node

import { Command, CommanderError } from 'commander';
import { readFileSync } from 'node:fs';
import { resolve } from 'node:path';

const DEFAULT_FILE = 'data.json';
const WEEK_TYPES = ['numerator', 'denominator'];

class CliError extends Error {
  constructor(message, exitCode = 1) {
    super(message);
    this.name = 'CliError';
    this.exitCode = exitCode;
  }
}

function formatHelp(command, helper) {
  const rows = (items, term, description) => {
    if (items.length === 0) return '';
    const terms = items.map(term);
    const width = Math.max(...terms.map((item) => item.length));
    return items
      .map((item, index) => ` ${terms[index].padEnd(width)}
${description(item)} `)
      .join('\n');
  };

  const sections = [];
  sections.push(` Використання: ${helper.commandUsage(command)} `);

  const description = helper.commandDescription(command);
  if (description) sections.push(description);

  const args = helper.visibleArguments(command);
  if (args.length > 0) {
    sections.push(
      `Аргументи:\n${rows(
        args,
        (arg) => helper.argumentTerm(arg),
        (arg) => helper.argumentDescription(arg),
      )} `
    );
  }

  const options = helper.visibleOptions(command);
  if (options.length > 0) {
    sections.push(
      `Опції:\n${rows(
        options,
        (option) => helper.optionTerm(option),
        (option) => helper.optionDescription(option),
      )} `
    );
  }

  const commands = helper.visibleCommands(command);
  if (commands.length > 0) {
    sections.push(
      `Команди:\n${rows(

```

```

        commands,
        (subcommand) => helper.subcommandTerm(subcommand),
        (subcommand) => helper.subcommandDescription(subcommand),
    )}` ,
    );
}

return `${sections.join('\n\n')}\n`;
}

function translateCommanderError(error) {
    const message = error.message ?? '';
    let match;

    if ((match = message.match(/missing required argument '([^']+)'/))) {
        return `Пропущено обов'язковий аргумент <${match[1]}>.`;
    }
    if ((match = message.match(/unknown option '([^']+)'/))) {
        return `Невідома опція «${match[1]}».`;
    }
    if ((match = message.match(/unknown command '([^']+)'/))) {
        return `Невідома команда «${match[1]}».`;
    }
    if ((match = message.match(/option '([^']+) ' argument missing/))) {
        return `Для опції «${match[1]}» не вказано значення.`;
    }
    if (message.includes('too many arguments')) {
        return 'Передає забагато аргументів.';
    }

    return message.replace(/^error:\s*/i, '') || 'Некоректний виклик програми.';
}

function isObject(value) {
    return value !== null && typeof value === 'object' &&
    !Array.isArray(value);
}

function validateData(data) {
    if (!isObject(data)) {
        throw new CliError('Корінь JSON-документа має бути об'єктом.');
```

```

        throw new CliError(`${location} повинен містити ціле «pair» і рядок
«time».`);
    }

    for (const weekType of WEEK_TYPES) {
        const lesson = slot[weekType];
        if (lesson === null) continue;
        if (!isObject(lesson)) {
            throw new CliError(`${location}.${weekType} має бути об'єктом або
null.`);
        }

        for (const field of ['subject', 'type', 'teacher', 'room']) {
            if (typeof lesson[field] !== 'string') {
                throw new CliError(`${location}.${weekType}.${field} має бути
рядком.`);
            }
        }
        if (typeof lesson.isRemote !== 'boolean') {
            throw new CliError(`${location}.${weekType}.isRemote має бути
логічним значенням.`);
        }
    }
    });
});

return data;
}

function loadData(filePath) {
    const absolutePath = resolve(process.cwd(), filePath);
    let text;

    try {
        text = readFileSync(absolutePath, 'utf8');
    } catch (error) {
        if (error.code === 'ENOENT') {
            throw new CliError(`Файл «${filePath}» не знайдено.`);
        }
        if (error.code === 'EISDIR') {
            throw new CliError(`Шлях «${filePath}» вказує на теку, а не на файл.`);
        }
        throw new CliError(`Не вдалося прочитати файл «${filePath}»:
${error.message}`);
    }

    let data;
    try {
        data = JSON.parse(text);
    } catch (error) {
        throw new CliError(`Файл «${filePath}» містить некоректний JSON:
${error.message}`);
    }

    return validateData(data);
}

апострофа. */
function normalize(value) {
    return String(value)
        .trim()
        .toLocaleLowerCase('uk-UA')
        .replace(/['`]/g, '"');
}

```

```

function findDay(data, requestedDay) {
    const day = data.schedule.find((item) => normalize(item.day) ===
normalize(requestedDay));
    if (!day) {
        const available = data.schedule.map((item) => item.day).join(', ');
        throw new CliError(`День «${requestedDay}» не знайдено. Доступні дні:
${available}.`, 4);
    }
    return day;
}

function parsePositiveInteger(value, optionName) {
    if (!/^\d+$/.test(String(value)) || Number(value) < 1 ||
!Number.isSafeInteger(Number(value))) {
        throw new CliError(`Опція ${optionName} очікує додатне ціле число,
отримано «${value}».`, 2);
    }
    return Number(value);
}

function parseWeekType(value, allowBoth = false) {
    const normalized = normalize(value);
    const allowed = allowBoth ? [...WEEK_TYPES, 'both'] : WEEK_TYPES;
    if (!allowed.includes(normalized)) {
        const names = allowBoth ? 'numerator, denominator або both' : 'numerator
або denominator';
        throw new CliError(`Невідомий тип тижня «${value}». Використайте
${names}.`, 2);
    }
    return normalized;
}

function isSameLesson(first, second) {
    return first !== null && second !== null && JSON.stringify(first) ===
JSON.stringify(second);
}

function weekLabel(weekType) {
    return {
        numerator: 'чисельник',
        denominator: 'знаменник',
        weekly: 'щотижня',
    }[weekType];
}

function printHeader(data) {
    console.log(`Група: ${data.group} | Курс: ${data.course}`);
}

function printLesson(dayName, slot, lesson, type) {
    const format = lesson.isRemote ? 'дистанційно' : 'очно';
    console.log(`${dayName} | ${slot.pair} пара | ${slot.time} |
${weekLabel(type)}`);
    console.log(`    ${lesson.subject} (${lesson.type}); ${lesson.teacher}; ауд.
${lesson.room}; ${format}`);
}

/** Повертає заняття без дублювання однакової щотижневої пари. */
function lessonOccurrences(day) {
    const result = [];

    for (const slot of day.lessons) {
        if (isSameLesson(slot.numerator, slot.denominator)) {
            result.push({ day, slot, lesson: slot.numerator, weekType: 'weekly' });
        }
    }
}

```

```

        continue;
    }
    for (const weekType of WEEK_TYPES) {
        if (slot[weekType] !== null) {
            result.push({ day, slot, lesson: slot[weekType], weekType });
        }
    }
}

return result;
}

/** Читає вкладене поле за шляхом на зразок lessons.0.numerator.subject. */
function getByPath(object, path) {
    const parts = String(path).split('.');
    if (parts.length === 0 || parts.some((part) => part === '')) {
        throw new CliError('Шлях до поля не може бути порожнім і не повинен містити порожніх частин.', 2);
    }

    let current = object;
    for (const part of parts) {
        if ((isObject(current) || Array.isArray(current)) && Object.hasOwn(current, part)) {
            current = current[part];
        } else {
            throw new CliError(`Поле за шляхом «${path}» не існує.`, 4);
        }
    }
    return current;
}

function printValue(value) {
    if (typeof value === 'string') {
        console.log(value);
    } else {
        console.log(JSON.stringify(value, null, 2));
    }
}

const program = new Command();

program
    .name('schedule-cli')
    .description('CLI-програма для роботи з розкладом занять групи (варіант 1).')
    .version('1.0.0', '-V, --version', 'Показати версію програми')
    .helpOption('-h, --help', 'Показати довідку')
    .addHelpCommand('help [command]', 'Показати довідку для команди')
    .option('-f, --file <path>', 'Шлях до JSON-файлу з розкладом',
    DEFAULT_FILE)
    .configureHelp({ formatHelp })
    .configureOutput({
        writeOut: (text) => process.stdout.write(text),
        // Стандартні англomовні помилки приховано; нижче вони перекладаються.
        writeErr: () => {},
    })
    .exitOverride();

// стислий перелік основних елементів.
program
    .command('list')
    .description('Показати стислий перелік днів розкладу')
    .option('-l, --limit <number>', 'Обмежити кількість показаних днів')

```

```

.action((options) => {
  const data = loadData(program.opts().file);
  const limit = options.limit === undefined
    ? data.schedule.length
    : parsePositiveInteger(options.limit, '--limit');
  const days = data.schedule.slice(0, limit);

  printHeader(data);
  console.log(`Показано днів: ${days.length} із ${data.schedule.length}`);
  days.forEach((day, index) => {
    const numeratorCount = day.lessons.filter((slot) => slot.numerator !==
null).length;
    const denominatorCount = day.lessons.filter((slot) => slot.denominator
!== null).length;
    console.log(
      `${index + 1}. ${day.day}: часових слотів - ${day.lessons.length}, `
      + `чисельник - ${numeratorCount}, знаменник - ${denominatorCount}`,
    );
  });
});

program
  .command('show')
  .description('Показати повний JSON одного дня')
  .argument('<day>', 'Назва дня тижня, наприклад Понеділок')
  .action((dayName) => {
    const data = loadData(program.opts().file);
    printValue(findDay(data, dayName));
  });

program
  .command('field')
  .description('Показати поле дня за крапковим шляхом')
  .argument('<day>', 'Назва дня тижня')
  .argument('<path>', 'Шлях, наприклад lessons.0.numerator.subject')
  .action((dayName, path) => {
    const data = loadData(program.opts().file);
    const day = findDay(data, dayName);
    printValue(getByPath(day, path));
  });

// заняття за обраний день.
program
  .command('day')
  .description('Показати заняття за обраний день тижня')
  .argument('<day>', 'Назва дня тижня')
  .option('-w, --week <type>', 'Тип тижня: numerator, denominator або both',
'both')
  .action((dayName, options) => {
    const data = loadData(program.opts().file);
    const day = findDay(data, dayName);
    const selectedWeek = parseWeekType(options.week, true);
    const types = selectedWeek === 'both' ? WEEK_TYPES : [selectedWeek];

    printHeader(data);
    console.log(`День: ${day.day} | Режим: ${selectedWeek}`);
    let count = 0;
    for (const slot of day.lessons) {
      for (const weekType of types) {
        const lesson = slot[weekType];
        if (lesson !== null) {
          const label = isSameLesson(slot.numerator, slot.denominator) ?
'weekly' : weekType;

```



```

        if (label === 'weekly' && weekType === 'denominator' &&
types.length === 2) continue;
        printLesson(day.day, slot, lesson, label);
        count += 1;
    }
}
}
console.log(`Знайдено занять: ${count}`);
});

program
.command('teacher')
.description('Знайти заняття певного викладача')
.argument('<name>', 'Повне ім'я або його частина, наприклад Чмихало')
.option('-r, --remote', 'Показати лише дистанційні заняття')
.action((name, options) => {
    const data = loadData(program.opts().file);
    const query = normalize(name);
    const matches = data.schedule
        .flatMap(lessonOccurrences)
        .filter(({ lesson }) => normalize(lesson.teacher).includes(query))
        .filter(({ lesson }) => !options.remote || lesson.isRemote);

    if (matches.length === 0) {
        const suffix = options.remote ? ' серед дистанційних занять' : '';
        throw new CliError(`Занять викладача «${name}»${suffix} не знайдено.`);
    }

    printHeader(data);
    console.log(`Викладач: ${name}${options.remote ? ' | лише дистанційні' : ''}`);
    matches.forEach(({ day, slot, lesson, weekType }) => {
        printLesson(day.day, slot, lesson, weekType);
    });
    console.log(`Знайдено занять: ${matches.length}`);
});

// розклад чисельника/знаменника зі щотижневими парами.
program
.command('week')
.description('Показати розклад для чисельника або знаменника')
.argument('<type>', 'Тип тижня: numerator або denominator')
.option('-d, --day <day>', 'Обмежити розклад одним днем')
.action((type, options) => {
    const data = loadData(program.opts().file);
    const selectedWeek = parseWeekType(type);
    const days = options.day ? [findDay(data, options.day)] : data.schedule;

    printHeader(data);
    console.log(`Розклад: ${weekLabel(selectedWeek)}`);
    let count = 0;
    for (const day of days) {
        for (const slot of day.lessons) {
            const lesson = slot[selectedWeek];
            if (lesson !== null) {
                const label = isSameLesson(slot.numerator, slot.denominator) ?
'weekly' : selectedWeek;
                printLesson(day.day, slot, lesson, label);
                count += 1;
            }
        }
    }
    console.log(`Знайдено занять: ${count}`);

```

```

    });

    async function main() {
      try {
        if (process.argv.length === 2) {
          program.outputHelp();
          return;
        }
        await program.parseAsync(process.argv);
      } catch (error) {
        if (error instanceof CommanderError) {
          if (error.exitCode === 0) return;
          console.error(`Помилка: ${translateCommanderError(error)}`);
          console.error('Виконайте «node index.js --help», щоб переглянути довідку.');
          process.exitCode = error.exitCode || 2;
          return;
        }
        if (error instanceof CliError) {
          console.error(`Помилка: ${error.message}`);
          process.exitCode = error.exitCode;
          return;
        }

        console.error('Помилка: сталася непередбачена помилка.');
```

process.exitCode = 1;
 }
 }

 await main();